Delegates, Anony. Types and Anony. Function



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



Delegates

Anonymous Types

Anonymous Function

Delegates

- A function can have one or more parameters of different data types, but what if you want to pass a function itself as a parameter?
- A delegate is like a pointer to a function. It is a reference type data type and it holds the reference of a method.
- All the delegates are implicitly derived from **System.Delegate** class.
- There are three steps involved while working with delegates:
  - Declare a delegate
  - Set a target method
  - Invoke a delegate

Delegate Syntax:
<access modifier> delegate <return type> <delegate_name>(<parameters>)

Example: Declare delegate

```
public delegate void Print(int value);
```

Important Points About Delegates:

- Provides a good way to encapsulate the methods.
- Delegates are the library class in System namespace.
- These are the type-safe pointer of any method.
- Delegates are mainly used in implementing the call-back methods and events.
- Delegates can be chained together as two or more methods can be called on a single event.
- It doesn't care about the class of the object that it references.
- Delegates can also be used in "anonymous methods" invocation.
- Anonymous Methods(C# 2.0) and Lambda expressions(C# 3.0) are compiled to delegate types in certain contexts. Sometimes, these features together are known as anonymous functions.

```
class Program1
   {
      // declare delegate
      public delegate void Print(int value);

      static void Main(string[] args)
      {
         // Print delegate points to PrintD
         Print printDel = PrintID;

         // or
         // Print printDel = new Print(PrintID);

         printDel(10);
         printDel(20000);

         // Print delegate points to PrintSalary
         printDel = PrintSalary;

         printDel(11);
         printDel(25000);
         Console.ReadLine();
      }
```

```
public static void PrintID(int num)
      {
         Console.WriteLine("Your ID is: {0}", num);
      }

      public static void PrintSalary(int amount)
      {
         Console.WriteLine("Your Salary is: {0}", amount);
      }
   }
```

- **Predicate Delegate**

- A Predicate delegate is an in-built generic type **delegate**.
- It works with those methods which contain some set of criteria and determine whether the passed parameter fulfill the given criteria or not
- This delegate takes only **one input** and **returns the value in the form of true or false.**

```
public delegate bool Predicate <in P>(P obj);
```

```
{
internal class PredicateDelDemo
    {
        public bool ValidateLength(string myStr)
        {
            if (myStr.Length < 10)
            {
                return true;
            }
            else
            {
                return false;
            }
        }
        public bool ValidateEmpCode(Employee employee)
        {
            if (employee.EmpCode <= 0)
            {
                return false;
            }
            else
            {
                return true;
            }
        }
```

```
 static void Main()
        {
            PredicateDelDemo predicateDel = new
PredicateDelDemo();

            Predicate<string> predicate =
predicateDel.ValidateLength;
            var result = predicate("Hello World");
            Console.WriteLine(result);

            Console.WriteLine(result1);
        }
    }



}
```

- **Action Delegate**

- A Action is a built-in generic type delegate.
- This delegate saves you from defining a custom delegate.
- **It can contain minimum 1 and maximum of 16 input parameters and does not contain any output parameter**.
- The Action delegate is generally used for those methods which do not contain any return value, or in other words, Action delegate is used with those methods whose return type is void.

```
public delegate void Action < in P > (P obj);
```

```
namespace AdvanceConceptsDemo
{
internal class ActionDelDemo
    {
        public static void Add(int num1, int num2)
        {
            Console.WriteLine(num1 + num2);
        }

        static void Main()
        {

            Action<int, int> action = Add;
            action(20, 30);
        }
    }
}
```

- **Func Delegate**

- Func delegate is an in-built generic type delegate.
- This delegate saves you from defining a custom delegate .
- It can **contain minimum 0 and maximum of 16 input parameters** in it and **contain only one out parameter**.
- The last parameter of the Func delegate is the out parameter which is considered as return type and used for the result.
- Func is generally used for those methods which are **going to return a value**, or in other words, Func delegate is used for value returning methods.

```
public delegate void Func< int,int > (int,int);
```

```
namespace AdvanceConceptsDemo
{
internal class FuncDelDemo
    {
        public static int Add(int num1, int num2)
        {
            return num1 + num2;
        }

        static void Main()
        {
            Func<int, int, int> func = Add;
            var result = func(30, 20);
            Console.WriteLine($"The sum is:{result}");
        }
    }
}
```

Anonymous Types

- In C#, an anonymous type is a type (class) without any name that can contain **public read-only** properties only.
- It cannot contain other members, such as fields, methods, events, etc.
- You create an anonymous type using the new operator with an **object initializer syntax.**
- The **implicitly typed variable- var** is used to hold the reference of anonymous types.

```
var std = new { Id = 1, FirstName = "Scott", LastName = "Lee" };
```

```
var students = new[] {
new { Id = 1, FirstName = "Mark", LastName = "Zuckerberg" },
new { Id = 2, FirstName = "Steve", LastName = "Jobs" },
new { Id = 3, FirstName = "Bill", LastName = "Gates" }
};
```

C# - Anonymous Method

- As the name suggests, an anonymous method is a method without a name.
- Anonymous methods in C# can be defined using the delegate keyword and can be assigned to a variable of delegate type.

Example: Anonymous Method

```
public delegate void Print(int value);
static void Main(string[] args) {
Print print = delegate(int val) {
Console.WriteLine("Inside Anonymous method. Value: {0}", val);
};
print(100);
}
```

Example: Anonymous Method as Parameter

```
public delegate void Print(int value);

class Program {
        public static void PrintHelperMethod(Print printDel,int val)
        {
        val += 10;
        Print(val);
         }
        static void Main(string[] args)
        {
        PrintHelperMethod(delegate(int val)
        {
        Console.WriteLine("Anonymous method: {0}", val);
        }, 100);
        }
    }
```

Example: Anonymous function using lambda expression.

```
internal class AnonyMethodDemo
   {
static void Main()
      {

         // C# 3.0. A delegate can be initialized with
         // a lambda expression. The lambda also takes a int
         // as an input parameter (x). The type of x is inferred by the

         // How to use Action generic Delegate to create Anonymous function with lambda expression

         Action<int> actionDel = (x) => Console.WriteLine($"value using lambda expression with Action Delegate:{x}");
         actionDel(200);

         // How to use Func generic Delegate to create Anonymous function with lambda expression

         Func<int, int> funDel = (x) => ++x;

         var result = funDel(10);
         Console.WriteLine($"value using lambda expression with Func Delegate:{result}");

      }}
```

- **Anonymous Method Limitations**
  - It cannot contain jump statement like goto, break or continue.
  - It cannot access ref or out parameter of an outer method.

References

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/

https://www.tutorialsteacher.com/csharp/csharp-anonymous-method

https://www.geeksforgeeks.org/anonymous-method-in-c-sharp/

https://www.c-sharpcorner.com/article/anonymous-methods-and-lambda-expressions-in-c-sharp/

https://www.koderhq.com/tutorial/csharp/lambda

https://www.c-sharpcorner.com/article/anonymous-methods-and-lambda-expressions-in-c-sharp/

https://itnext.io/delegates-anonymous-methods-and-lambda-expressions-5ea4e56bbd05